

Plano de Aula – Introdução ao Pensamento Computacional

Objetivo da Aula:

- Os alunos devem compreender os 4 pilares do pensamento computacional: Decomposição, Reconhecimento de Padrões, Abstração e Pensamento Algorítmico.
- Os alunos devem ser capazes de traduzir problemas analíticos e de pesquisa de seus domínios originais em etapas lógicas e pseudo-códigos.
- Os alunos devem desenvolver e estruturar códigos funcionais através de programação em blocos (Google Blockly), focando na estrutura estática e no fluxo dinâmico sem a carga cognitiva da sintaxe.
- Os alunos devem ser capazes de testar e depurar seus algoritmos de forma colaborativa, mitigando o isolamento técnico e a frustração com o erro.

Conteúdo da aula (Duração Total: 4h):

Bloco 1: Apresentando o objetivo da Aula e Realizando a técnica pedagógica de Faça Agora! (40 min)

Apresentar o objetivo! (5 min)

- Hoje iremos aprender sobre o Pensamento Computacional e seus conceitos básicos de lógica de programação.
- O foco será aplicar o pensamento algorítmico para modelar problemas interdisciplinares reais das pesquisas de vocês, construindo fluxos lógicos e soluções através da ferramenta Google Blockly.
- **Compreenderemos que o erro e a depuração (debugging) fazem parte do método de pesquisa, distanciando-nos da ineficiente "tentativa e erro".**

Faça Agora! (15 min)

- **Desafio Inicial (Sem Tecnologia):** Como fazer um cuscuz?
- Descrevam em linguagem natural, utilizando uma folha de papel, a sequência lógica de passos para a solução, mapeando o que entra, como será o processo e o que é a saída.

Avisos iniciais: (2 min)

- Reforçar o ambiente colaborativo e seguro: os erros de lógica são tratados como etapas naturais da modelagem, minimizando a ansiedade e os estereótipos associados à área

de exatas.

Feedback do Faça Agora: (18 min)

- Após a tentativa de descrever a receita do cuscuz, vamos discutir a experiência em grupo:
 - **Dificuldades:**
 - Quais foram os principais obstáculos encontrados ao tentar detalhar cada passo para o "computador"?
 - **Pontos Ambíguos**
 - Que partes da descrição ficaram abertas a diferentes interpretações ou careceram de clareza?
 - **Acertos**
 - Onde a lógica aplicada foi precisa e o "algoritmo", mesmo que inicial, se mostrou eficaz?

Bloco 2: Apresentando o conceito de Pensamento Computacional e os seus 4 Pilares (60 min)

Pensamento Computacional (10 min)

- **Conceito Chave:** O que é o Pensamento Computacional, segundo Jeannette Wing, e sua aplicação como habilidade analítica não restrita a computadores.
- Pensamento Computacional é a habilidade de formular problemas e expressar soluções de forma tão clara que qualquer pessoa — ou máquina — consiga executá-las. Não é sobre programar: é sobre pensar com precisão.
- Desmistificando o Erro: O erro de lógica não é uma falha de talento — é uma etapa natural da modelagem científica e computacional.

Os Quarto Pilares do Pensamento Computacional (20 min)

- **Conceito Chave:** Os 4 Pilares do Pensamento Computacional
- Apresentação teórica focada na desmistificação e redução da barreira sociotécnica:
 - Decomposição de problemas complexos.
 - Reconhecimento de Padrões.
 - Abstração.
 - Variáveis, tipos de dados e estruturas de controle (if/else, loops).

Exercício Prático (Sem Tecnologia) após o conceito do Pensamento Computacional (20 min)

Faça Agora! - Média do Aluno



- **Novo Desafio (Sem Tecnologia): Calcular a Média e o Conceito Educacional do Aluno?**
- Descrevam em linguagem natural, utilizando uma folha de papel, a sequência lógica de passos para a solução, mapeando o que entra, como será o processo e o que é a saída.

Obs.: Exercício prático para ver a evolução das soluções, após o conceito de Pensamento Computacional.

Feedback do Faça Agora: (10 min)

- Após a tentativa de descrever o algoritmo de calcular a média e o conceito, vamos discutir a experiência em grupo:
 - **Dificuldades:**
 - Quais foram os principais obstáculos encontrados ao tentar detalhar cada passo para o "computador"?
 - **Pontos Ambíguos**
 - Que partes da descrição ficaram abertas a diferentes interpretações ou careceram de clareza?
 - **Acertos**
 - Onde a lógica aplicada foi precisa e o "algoritmo", mesmo que inicial, se mostrou eficaz?

Bloco 3: Andaimos Cognitivos (Método Parsons) e o Google Blockly (60 min)

Método Parsons: Aprender sem Medo (10 min)

- **Conceito Chave:** O que é o Método de Parsons, o foco é puramente no sequenciamento lógico, eliminando o medo da sintaxe complexa e permitindo que o aluno raciocine sobre estrutura
- **Conceito Chave:** Programação em pares, a colaboração em duplas não é apenas uma estratégia pedagógica — é uma prática real da indústria de tecnologia.

Apresentação do Blockly (5 min)

- **Conceito Chave:** O que é o Blockly, é uma ferramenta gratuita e open-source do Google que permite programar arrastando e encaixando blocos. Apresentar a plataforma Blockly Demo.

Práticas com o Blockly Demo (45 min)

- **Desafio Blockly: Calculando os Degraus da Escada. (15 min)**
 - Prática do Aluno (10 min)
 - Feedback dos educadores (5 min)

- Desafio Blockly: Média de Aluno - Calcule o Conceito (15 min)
 - Prática do Aluno (10 min)
 - Feedback dos educadores (5 min)
- Desafio Blockly: Tabuada Interativa (15 min)
 - Prática do Aluno (10 min)
 - Feedback dos educadores (5 min)

Bloco 4: Entender, Analisar e Testar uma Linguagem de Programação (Python) (60 min)

Introdução a Linguagem Python (60 min)

- Conceituando **Variáveis** com Python, linkando com o código já produzido com o Blockly.(15 min)
- Conceituando **Operadores** com Python, linkando com o código já produzido com o Blockly.(15 min)
- Conceituando **Estrutura Condicional** com Python, linkando com o código já produzido com o Blockly.(15 min)
- Conceituando **Estrutura de Repetição** com Python, linkando com o código já produzido com o Blockly.(15 min)

Bloco 5: Arremate e Fechamento (20 min)

Arremate: Praticando Python Básico (15 min)

- Exercício prático de média de notas e conceito realizando um arremate com tudo que já foi lecionado.

O que Construimos Hoje (2 min)

- Realiza um resumo do que foi aprendido ,e provocar os alunos sobre os conceitos e tecnologias apresentadas.

Próximos Passos: Continue Aprendendo (3 min)

- Continue praticando fora da sala de aula: cada pequena atividade ajuda a fortalecer sua lógica, sua autonomia e sua confiança para aprender programação.

Nota da Aula:

- **Arranjo da sala:** O ambiente físico deve seguir uma configuração colaborativa em formato "U" ou clusters (pequenos grupos) para viabilizar o Think-Pair-Share e a Tutoria Recíproca. Garantir espaço de circulação livre para monitoria ativa do instrutor.
- **Infraestrutura técnica:** Computadores individuais ou em pares conectados à internet,

projektor multimídia para as explicações e espaço de lousa/quadro branco com materiais de papelaria (post-its e canetas) para o design algorítmico offline.

- **Comunicação e Feedback:** Avaliação deve ser contínua com feedback imediato durante as etapas práticas. A linguagem da correção deve afastar julgamentos de "talento para exatas" e basear-se em analogias diretas com a área de formação (ex: química, negócios) do aluno.